College of Engineering

Software Engineering

Database Systems [SWEG2108]

Hotel Management System: Abyssinia Grand Hotel

Section B

Group members	ID no.
1. Hana Mekonnen	ETS0692/17
2. Helen Mulugeta	ETS0719/17
3. Helen Tamene	ETS0720/17
4. Hermela Tilahun	ETS0741/17
5. Hermon Ermias	ETS0742/17
6. Hilina Moges	ETS0748/17

Submitted to: Mr. Yaynshet Medhin

Submission date: May 15/ 2026.

Table of Contents

1. Introduction 1
1.1. Background of the Study 1
1.2. Problem Statement 1
1.3. Objectives 1
1.4. Scope of the Project 2
1.5. Methodology 2
1.6. Tools and Technologies 2
2. Database Environment and Physical Implementation 3
2.1. Database Management System Selection 3
2.2. Physical Schema Construction 3
2.3. Data Population and Load Testing 3
2.4. Conceptual and Logical Database Design 3
2.5. Logical Schema and Normalization Theory 4
2.6. Relational Mapping 5
3. Implementation and Result 6
3.1. MySQL Implementation (Structured Data) 6
3.2. MongoDB Implementation (Unstructured Data) 6
4. References 7
5. Annex 8
5.1. Annex A: System Design Inputs (Forms) 8
5.2. Annex B: Survey Questions (Requirement Analysis) 10
5.3. Annex C: SQL Source Code 11
5.4. Annex D: MongoDB Source Code 17

1. Introduction

1.1. Background of the Study

In the modern hospitality industry, data is the most valuable asset for operational efficient and guest attraction. Abyssinia Grand Hotel is a premier establishment that provides diverse lodging and auxiliary services. As the hotel scales, managing guest records, room availability, service usage (spa, dining, laundry), and complex billing through manual or fragmented systems has become unsustainable. An integrated database approach using both Relational (MySQL) and NoSQL (MongoDB) technologies is essential to ensure data integrity, scalability and personalized guest experiences.

1.2. Problem Statement

The current information management strategy at Abyssinia Grand Hotel faces several critical challenges.

Manual tracking of room status and service orders leads to delays and human errors during the check-in and check-out processes. Non-normalized data storage results in update anomalies, where changes to room rates or service costs are not reflected across all historical records, violating the BCNF principle.

The lack of a unified link between service usage and the final billing entity often results in unbilled services, directly impacting the hotel's profitability. Transactional data is separated from guest preference data, making it impossible to perform meaningful analytics on guest behavior or loyalty.

1.3. Objectives

General Objective

To design, develop and implement an integrated high-performance database management system that synchronizes room reservations, service consumption and financial transactions while maintaining high data integrity and flexibility.

Specific Objectives

- To perform a detailed requirement analysis and conceptualize the system using Entity-Relationship Diagram (ERD).
- To implement a Relational Schema normalized to BCNF to eliminate data redundancy in transactional records.

- To deploy a NoSQL (MongoDB) solution for storing unstructured guest feedback and varied service performances.

1.4. Scope of the Project

The project is bounded by the following functional modules.

1. Guest management: Registration, contact history and multi-valued phone number management.
2. Inventory and accommodation: categorization of rooms by type, price management and real-time tracking.
3. Transactional life-cycle: handling the complete flow from reservation and check-in to service utilization and final payment.
4. Service integration: a dedicated module for auxiliary hotel services (gym, spa,laundry) and their associated costs.
5. Reporting: generation of consolidated bills and basic management summaries.

1.5. Methodology

- The project follows a structured database life cycle approach.
- Database initial study - analyzing the hotel's operations and identifying the entities and relationships through simulated interviews.
- Database design - conceptual design for creating the ERD and logical design for mapping the ERD to s relational model and applying normalization.
- Implementation - writing data definition language scripts for MySQL. Populating the database with at least 20 guest records for testing and designing the MongoDB collections for flexible data types.
- Testing and validation - running CRUD operations to ensure the system handles data correctly without anomalies.

1.6. Tools and Technologies

- Modeling: Lucid chart or Draw.io for ERD and Logical schema visualization.
- Relational engine: MySQL Community Server and Workbench for structured transactional data.
- NoSQL engine: MongoDB Compass for document-oriented storage of guest preferences.

2. Database Environment and Physical Implementation

2.1. Database Management System Selection

The core of the Abyssinia Grand Hotel system is built upon the MySQL 8.0 relational engine. This DBMS was selected for its high-performance indexing capabilities and its strict adherence to the ACID (Atomicity, Consistency, Isolation, Durability) properties, which are essential for handling financial transactions and guest reservations. The implementation utilizes the InnoDB storage engine to support row-level locking and muti-version concurrency control, ensuring that room availability remains accurate during high traffic booking periods.

2.2. Physical Schema Construction

The implementation phase translating the logical design into a physical reality using Data Definition Language (DDL). we enforced domain integrity by using specific data types, such as DECIMAL (10,2) for financial precision in pavements and TIMESTAMP for real time tracking of transactions.

Physical links were established between tables using Primary Keys and Foreign Keys. We implemented referential integrity constraints to prevent orphan records for like ensuring pavement can't exist without a corresponding BookingID. The system uses AUTO_INCREMENT sequences for all primary identifiers, ensuring unique system generated keys for every new guest and reservation.

2.3. Data Population and Load Testing

To stimulate the operational environment of a luxury hotel, the DBMS was populated with a robust data-set using Data Manipulation Language (DML). around 10 unique records were inserted across the schema to test the relationship constraints and query performance. This population includes a diverse mix of room statuses (available, occupied, under maintenance) and guest histories, allowing us to verify that the DBMS correctly manages complex join operations and data retrieval.

2.4. Conceptual and Logical Database Design

Conceptual Design (ER Analysis)

The conceptual design represents the high-level business logic of the Abyssinia Grand Hotel. By identifying the essential entities and their relationships, we mapped the hotel's operational workflow into a formal model.

Entities:

- Guest: the primary stakeholder, defined by unique identity attributes.
- Room & Room Type: separated to allow for categorical management of hotel inventory.
- Booking and Payment: the central associative entity that captures the temporal relationship between guests and hotel rooms and the entity that is totally dependent on the former respectively.
- Service and Service Usage: specialized entities designed to track extra entities consumed during a guest's stay.

The relationship cardinality was carefully defined; one room type can categorize multiple rooms and one guest can initiate multiple bookings over time.

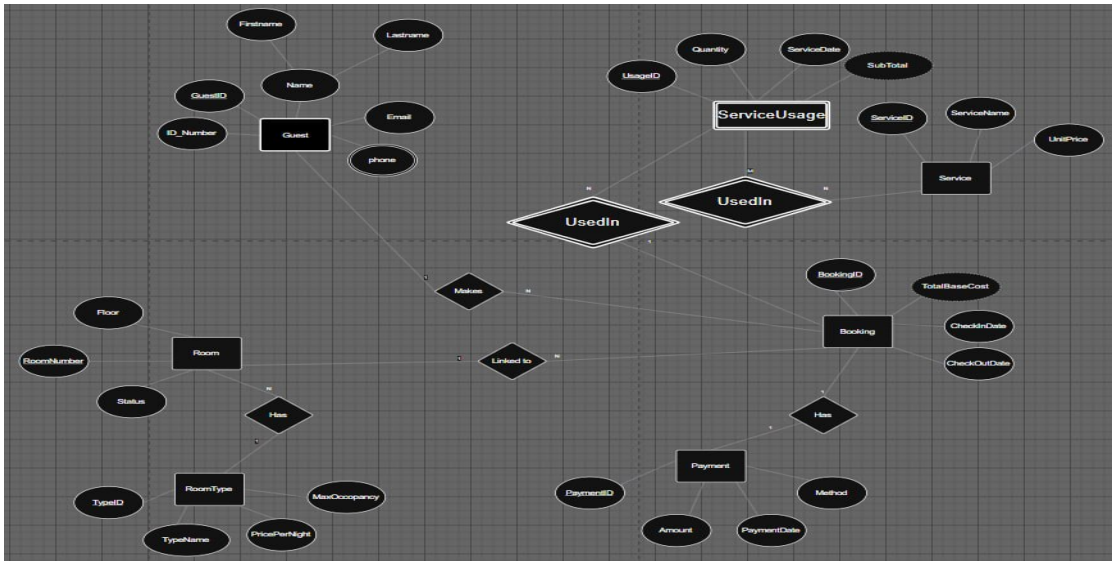


Figure 2.1 Hotel Management System ER Diagram

2.5. Logical Schema and Normalization Theory

The logical architecture was refined through the process of normalization to reach Boyce-Codd Normal Form (BCNF). this ensures database is free from insertion, update and deletion anomalies.

In the first normal form (1NF) we eliminated multi-valued attributes, specifically by moving Guest phone numbers into a dedicated GuestPhone table, ensuring each cell contains only atomic values.

In the second normal form (2NF) and third normal form (3NF) we removed partial and transitive dependencies. Attributes like PricePerNight were moved to the RoomType table. This ensures that a price change only occurs in one location,

preventing data inconsistency across Room table. Every determinant in our relation schema is a candidate key. This design guarantees that the database is structurally sound and optimized for storage efficiency.

2.6. Relational Mapping

The final logic design consists of 7 normalized tables and one for phone number table. We utilized foreign key mapping to enforce the business rules of the hotel.

RoomType-->Room: links physical units to their pricing and occupancy rules.

Guest-->Booking: tracks which individual is responsible for a reservation.

Booking-->Payment: ensures every financial transaction is tied to a specific stay.

Service-->ServiceUsage: monitors the consumption of hotel facilities.

We implemented on update cascade to ensure that any changes to primary identifiers are automatically reflected throughout the related tables, maintaning a self-consistent data state at all times.

Fig: Relational Mapping

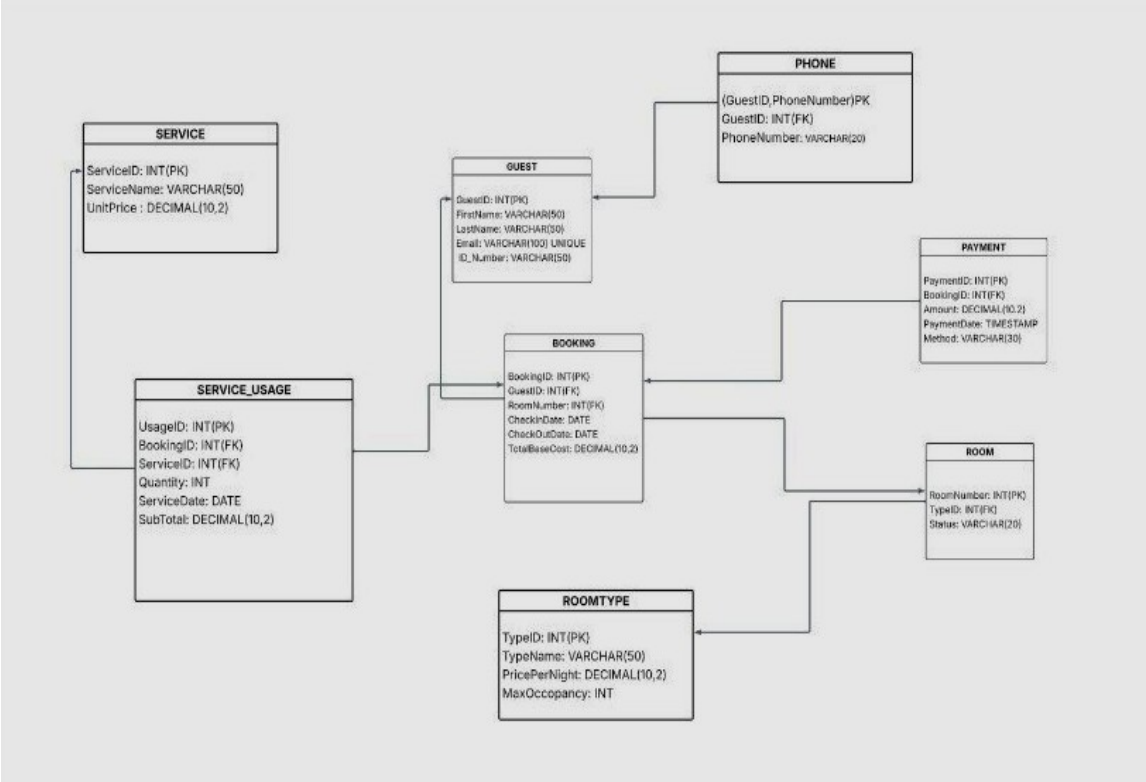


Figure 2.2Hotel Management System Relational Mapping

3. Implementation and Result

3.1. MySQL Implementation (Structured Data)

We successfully implemented the relational schema in MySQL 8.0. each table was created with appropriate constraints (primary keys, foreign keys and not null) to ensure data integrity.

We used decimal types for pricing to avoid rounding errors during check-out. A join query between guest and booking correctly displays all active reservations.

3.2. MongoDB Implementation (Unstructured Data)

As per the requirement for dual-DBMS approach, we utilized MongoDB to handle guest documents and flexible service logs. This allows the hotel to store special requests or ID scans without needing a rigid table structure for every minor detail.

Bugs and Improvements

During the implementation phase, we encountered and resolved several issues.

Bug	Fix
Initial desktop junk files in the repository	Cleaned the root directory and implement a strict folder hierarchy (/mysql, /diagrams, /docs)
Data redundancy in the room table	Normalized the schema to BCNF by separating room type

Figure 4.1 Bugs and Improvements

Improvement: Added on delete cascade to bookings to ensure that when a guest record is removed, the system doesn't retain orphan records, thereby maintaining strict referential integrity across the database.

4. References

1. R.Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7thed. Boston, MA, USA: Pearson, 2017.

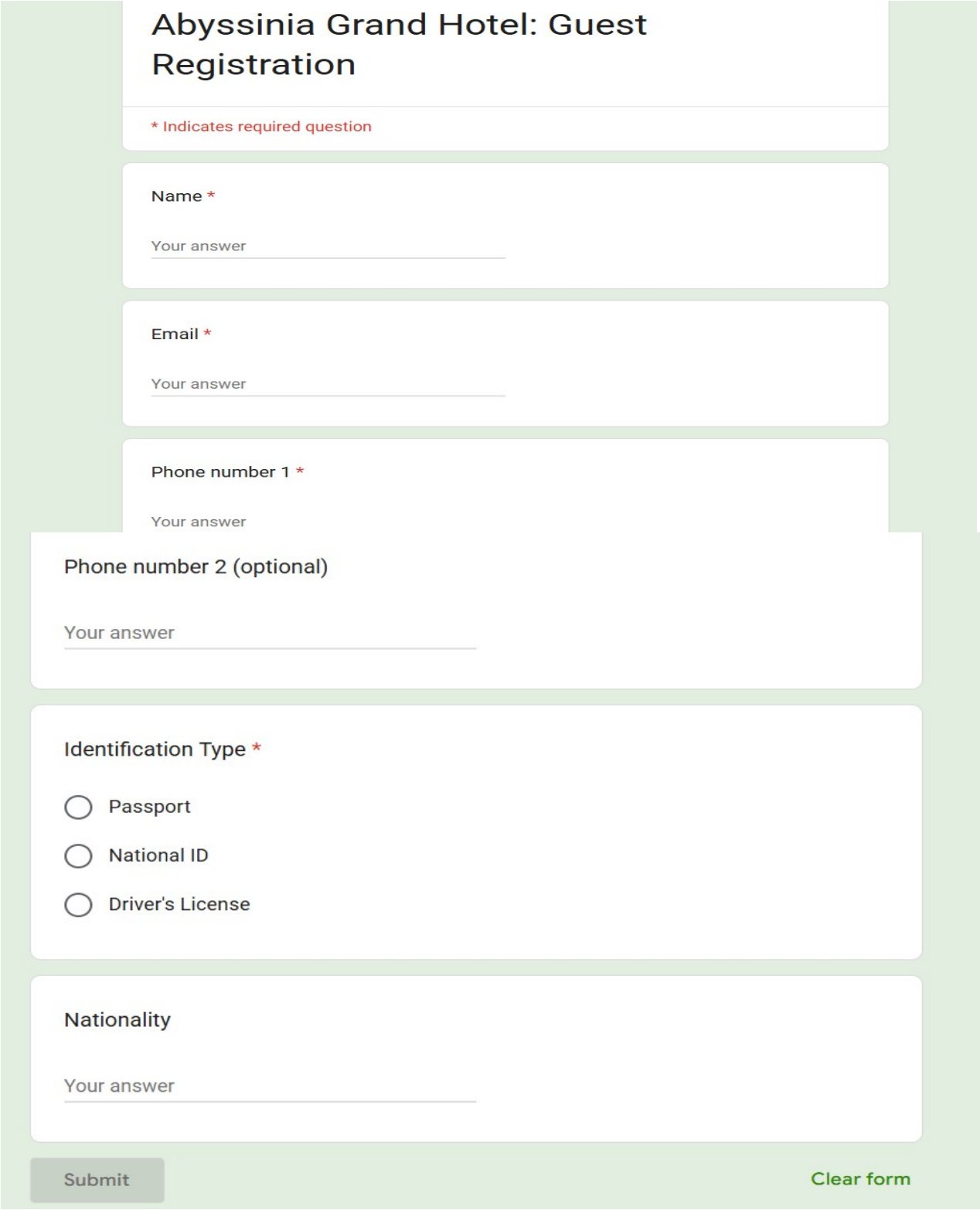
2. MySQL Documentation Team, “MySQL 8.0 Reference Manual,” Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

5. Annex

5.1. Annex A: System Design Inputs (Forms)

Form 1: Guest Registration Card

Used to populate the guest and guest phone tables.



Abyssinia Grand Hotel: Guest Registration

* Indicates required question

Name *

Your answer

Email *

Your answer

Phone number 1 *

Your answer

Phone number 2 (optional)

Your answer

Identification Type *

- Passport
- National ID
- Driver's License

Nationality

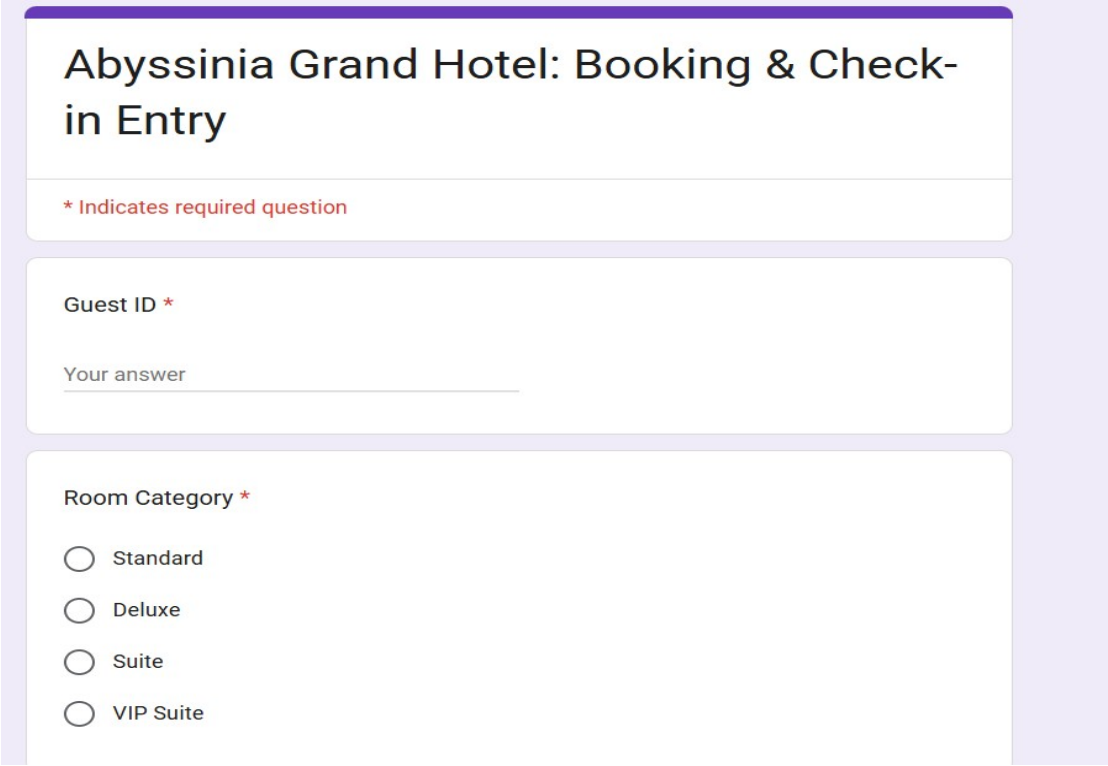
Your answer

Submit

Clear form

Form 2: Room Check-in/Check-out Slip

Used to populate the booking and room status tables



Abyssinia Grand Hotel: Booking & Check-in Entry

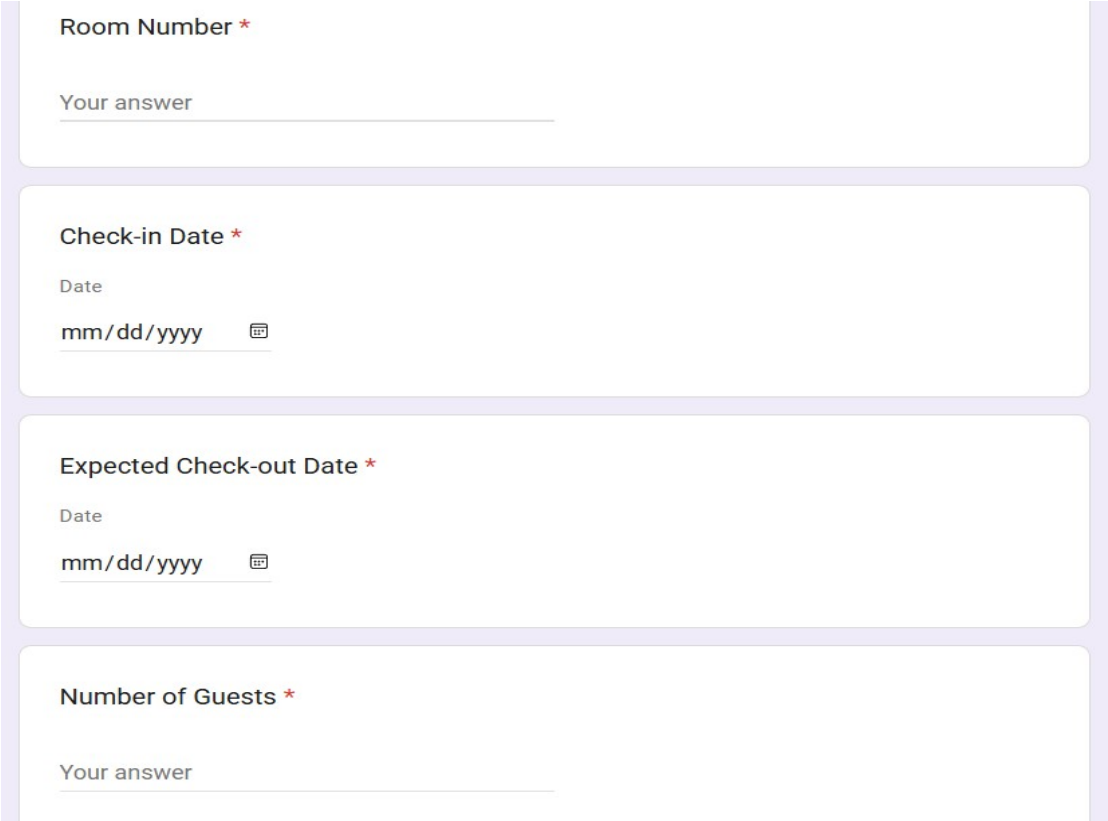
* Indicates required question

Guest ID *

Your answer

Room Category *

- Standard
- Deluxe
- Suite
- VIP Suite



Room Number *

Your answer

Check-in Date *

Date

mm/dd/yyyy

Expected Check-out Date *

Date

mm/dd/yyyy

Number of Guests *

Your answer

Form 1:
https://docs.google.com/forms/d/e/1FAIpQLSfNELFW36Goj9j1ZmRXyKFjJ5mJcoKxAYT2H0_pfIXTvGFczw/viewform?usp=publish-editor
Form 2:
https://docs.google.com/forms/d/e/1FAIpQLScsPQqnzQwA7ogHcslZqH6yW_8PeoNjy3pn3lBfOXbSJgj_eA/viewform?usp=publish-editor

5.2. Annex B: Survey Questions (Requirement Analysis)

These questions were used during the phase one analysis to understand the hotel's needs.

1. How do you currently track which rooms are cleaned and ready for new guests?
--> To design the status attribute in the room table
2. What extra services are most frequently used by guests?
--> To define the records in the service table.
3. Do you require a record of the payment method for every booking?
--> To design the method attribute in payment table.
4. Would you prefer to receive your booking confirmation via email?
--> To justify the email attribute in the guest table.
5. What is the maximum number of days a guest usually stays at Abyssinia Grand?
--> To set the appropriate data types for data ranges.
6. Do you offer seasonal discounts or corporate rates that change frequently?
--> To justify why price is in a separate room type table (normalization) rather than hard coded.
7. How often do you encounter double-booking errors?
--> To highlight the importance of primary key and unique constraints in the booking table.
8. How long should the system keep canceled booking records before archiving them?
--> To design the status attribute logic for the booking entity.

Results: Based on the survey results from the survey, 80% of staff reported difficulty tracking room availability. Our implementation of the status column in the room table directly solves this by providing real-time availability updates.

5.3. Annex C: SQL Source Code

1. MySQL Schema

```
CREATE DATABASE IF NOT EXISTS AbyssiniaGrandHotel;
USE AbyssiniaGrandHotel;

CREATE TABLE RoomType (
   TypeID INT AUTO_INCREMENT PRIMARY KEY,
   TypeName VARCHAR(50) NOT NULL,
   PricePerNight DECIMAL(10, 2) NOT NULL,
   MaxOccupancy INT NOT NULL
);

CREATE TABLE Guest (
   GuestID INT AUTO_INCREMENT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   ID_Number VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE GuestPhone (
   PhoneID INT AUTO_INCREMENT PRIMARY KEY,
   GuestID INT,
   PhoneNumber VARCHAR(20) NOT NULL,
   FOREIGN KEY (GuestID) REFERENCES Guest(GuestID) ON DELETE CASCADE
);

CREATE TABLE Room (
   RoomNumber INT PRIMARY KEY,
   TypeID INT,
   Status ENUM('Available', 'Occupied', 'Maintenance') DEFAULT 'Available',
   Floor INT,
   FOREIGN KEY (TypeID) REFERENCES RoomType(TypeID)
);

CREATE TABLE Booking (
```

```
    BookingID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT,
    RoomNumber INT,
    CheckIn DATE NOT NULL,
    CheckOut DATE NOT NULL,
    TotalBaseCost DECIMAL(10, 2),
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID),
    FOREIGN KEY (RoomNumber) REFERENCES Room(RoomNumber)
);

CREATE TABLE Service (
    ServiceID INT AUTO_INCREMENT PRIMARY KEY,
    ServiceName VARCHAR(100) NOT NULL,
    UnitPrice DECIMAL(10, 2) NOT NULL
);

CREATE TABLE ServiceUsage (
    UsageID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT,
    ServiceID INT,
    Quantity INT DEFAULT 1,
    ServiceDate DATE,
    SubTotal DECIMAL(10, 2),
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE
CASCADE,
    FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID)
);

CREATE TABLE Payment (
    PaymentID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT,
    Amount DECIMAL(10, 2) NOT NULL,
    Method ENUM('Cash', 'Credit Card', 'Telebirr', 'Bank Transfer'),
    PaymentDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);
```

```
INSERT INTO RoomType (TypeName, PricePerNight, MaxOccupancy) VALUES
('Standard', 2500.00, 2),
('Deluxe', 4500.00, 2),
('Presidential Suite', 12000.00, 4);

INSERT INTO Guest (FirstName, LastName, Email, ID_Number) VALUES
('Abebe', 'Bikila', 'abebe@email.com', 'ET001'),
('Sara', 'Mulugeta', 'sara@email.com', 'ET002'),
('Dawit', 'Isaak', 'dawit@email.com', 'ET003'),
('Mulu', 'Tesfaye', 'mulu@email.com', 'ET004'),
('Helen', 'Berhe', 'helen@email.com', 'ET005'),
('Yonas', 'Alemu', 'yonas@email.com', 'ET006'),
('Tigist', 'Assefa', 'tigist@email.com', 'ET007'),
('Elias', 'Girma', 'elias@email.com', 'ET008'),
('Rahel', 'Solomon', 'rahel@email.com', 'ET009'),
('Kidus', 'Yilma', 'kidus@email.com', 'ET010');

INSERT INTO GuestPhone (GuestID, PhoneNumber) VALUES
(1, '0911223344'), (1, '0922112233'),
(2, '0933445566'),
(3, '0944556677');

INSERT INTO Room (RoomNumber, TypeID, Status, Floor) VALUES
(101, 1, 'Available', 1),
(102, 1, 'Occupied', 1),
(201, 2, 'Available', 2),
(202, 2, 'Occupied', 2),
(301, 3, 'Occupied', 3);

INSERT INTO Service (ServiceName, UnitPrice) VALUES
('Spa Treatment', 1500.00),
('Gym Access', 500.00),
('Laundry Service', 300.00),
('Airport Pickup', 1000.00),
('Buffet Breakfast', 600.00);
```

```
INSERT INTO Booking (GuestID, RoomNumber, CheckIn, CheckOut, TotalBaseCost)
VALUES
(1, 101, '2026-05-01', '2026-05-03', 5000.00),
(2, 102, '2026-05-02', '2026-05-05', 7500.00),
(3, 201, '2026-05-05', '2026-05-07', 9000.00),
(4, 202, '2026-05-08', '2026-05-10', 9000.00),
(5, 301, '2026-05-10', '2026-05-12', 24000.00),
(6, 102, '2026-05-15', '2026-05-17', 5000.00);

INSERT INTO ServiceUsage (BookingID, ServiceID, Quantity, ServiceDate, SubTotal)
VALUES
(1, 1, 1, '2026-05-01', 1500.00),
(1, 5, 2, '2026-05-02', 1200.00),
(2, 4, 1, '2026-05-02', 1000.00),
(5, 1, 1, '2026-05-11', 1500.00);

INSERT INTO Payment (BookingID, Amount, Method) VALUES
(1, 7700.00, 'Telebirr'),
(2, 9000.00, 'Credit Card'),
(5, 27900.00, 'Credit Card');
```

2. MySQL CRUD and Testing

```
-- 1. CREATE: Add a new VIP Guest
INSERT INTO Guest (FirstName, LastName, Email, ID_Number)
VALUES ('Lidya', 'Solomon', 'lidya@email.com', 'ET012');

-- 2. UPDATE: Mark a room for cleaning (Maintenance)
UPDATE Room SET Status = 'Maintenance' WHERE RoomNumber = 101;

-- 3. DELETE: Remove an old payment record (ID 7)
DELETE FROM Payment WHERE PaymentID = 7;

-- 4. READ: Show the final state of all Bookings and Rooms
SELECT
   B.BookingID,
   G.FirstName,
```

```
    G.LastName,
    R.RoomNumber,
    R.Status as Room_Status
FROM Booking B
JOIN Guest G ON B.GuestID = G.GuestID
JOIN Room R ON B.RoomNumber = R.RoomNumber;
```

3. MySQL Result Screenshots

SCHEMAS

Filter objects

- abyssiniagrandhotel
  - Tables
    - booking
      - Columns
        - BookingID
        - GuestID
        - RoomNumbe
        - CheckIn
        - CheckOut
        - TotalBaseCos
      - Indexes
      - Foreign Keys
      - Triggers
    - guest
    - guestphone
    - payment
    - room
    - roomtype
    - service
    - serviceusage
  - Views
  - Stored Procedures
  - Functions
- globallms

Administration | Schemas

Information

Column: BookingID

Definition:
BookingID int AI PK

Object Info | Session

Limit to 1000 rows

```
CREATE DATABASE IF NOT EXISTS AbyssiniaGrandHotel;
USE AbyssiniaGrandHotel;

CREATE TABLE RoomType (
    TypeID INT AUTO_INCREMENT PRIMARY KEY,
    TypeName VARCHAR(50) NOT NULL,
    PricePerNight DECIMAL(10, 2) NOT NULL,
    MaxOccupancy INT NOT NULL
);

CREATE TABLE Guest (
    GuestID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    ID_Number VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE GuestPhone (
    PhoneID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT,
    PhoneNumber VARCHAR(20) NOT NULL,
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID) ON DELETE CASCADE
```

Output

Action Output

#	Time	Action	Message
1	18:33:19	drop database IF exists abyssiniagrandhotel	8 row(s) affected
2	18:37:31	CREATE DATABASE IF NOT EXISTS AbyssiniaGrandHotel	1 row(s) affected
3	18:37:31	USE AbyssiniaGrandHotel	0 row(s) affected
4	18:37:31	CREATE TABLE RoomType (TypeID INT AUTO_INCREMENT PRIMARY KEY, TypeName VARCHAR(5...	0 row(s) affected
5	18:37:31	CREATE TABLE Guest (GuestID INT AUTO_INCREMENT PRIMARY KEY, FirstName VARCHAR(50) NO...	0 row(s) affected
6	18:37:31	CREATE TABLE GuestPhone (PhoneID INT AUTO_INCREMENT PRIMARY KEY, GuestID INT, Phon...	0 row(s) affected

Limit to 1000 rows

```
-- 1. CREATE: Add a new VIP Guest
INSERT INTO Guest (FirstName, LastName, Email, ID_Number)
VALUES ('Lidya', 'Solomon', 'lidya@email.com', 'ET012');

-- 2. UPDATE: Mark a room for cleaning (Maintenance)
UPDATE Room SET Status = 'Maintenance' WHERE RoomNumber = 101;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BookingID	FirstName	LastName	RoomNumber	Room_Status
1	Abebe	Bikila	101	Maintenance
2	Sara	Mulugeta	102	Occupied
6	Yonas	Alemu	102	Occupied
3	Dawit	Isaak	201	Available
4	Mulu	Tesfaye	202	Occupied
5	Helen	Berhe	301	Occupied

Result Grid | Form Editor | Field Types

Result 1 | Read Only

Output

Action Output

#	Time	Action	Message
17	18:37:31	INSERT INTO Booking (GuestID, RoomNumber, CheckIn, CheckOut, TotalBaseCost) VALUES (1, 101, '2026...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0
18	18:37:31	INSERT INTO ServiceUsage (BookingID, ServiceID, Quantity, ServiceDate, SubTotal) VALUES (1, 1, 1, '202...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
19	18:37:31	INSERT INTO Payment (BookingID, Amount, Method) VALUES (1, 7700.00, 'Telebirr'), (2, 9000.00, 'Credit Ca...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
20	18:41:14	SELECT * FROM Guest LIMIT 0, 1000	10 row(s) returned
21	18:46:25	INSERT INTO Guest (FirstName, LastName, Email, ID_Number) VALUES ('Lidya', 'Solomon', 'lidya@email.com...	1 row(s) affected
22	18:46:25	UPDATE Room SET Status = 'Maintenance' WHERE RoomNumber = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
23	18:46:25	DELETE FROM Payment WHERE PaymentID = 7	0 row(s) affected
24	18:46:25	SELECT B.BookingID, G.FirstName, G.LastName, R.RoomNumber, R.Status as Room_Status ...	6 row(s) returned

5.4. Annex D: MongoDB Source Code

1. Collection schema


```
[
  {
    "GuestID": 1,
    "FirstName": "Abebe",
    "LastName": "Bikila",
    "Email": "abebe@email.com",
    "ID_Number": "ET001",
    "PhoneNumbers": ["0911223344", "0922112233"],
    "LoyaltyStatus": "Gold"
  },
  {
    "GuestID": 2,
    "FirstName": "Sara",
    "LastName": "Mulugeta",
    "Email": "sara@email.com",
    "ID_Number": "ET002",
    "PhoneNumbers": ["0933445566"],
    "LoyaltyStatus": "Silver"
  },
  {
    "GuestID": 3,
    "FirstName": "Dawit",
    "LastName": "Isaak",
    "Email": "dawit@email.com",
    "ID_Number": "ET003",
    "PhoneNumbers": ["0944556677"]
  },
  {
    "GuestID": 4,
    "FirstName": "Mulu",
    "LastName": "Tesfaye",
    "Email": "mulu@email.com",
    "ID_Number": "ET004",
    "PhoneNumbers": ["0955667788"]
```

},
{
"GuestID": 5,
"FirstName": "Helen",
"LastName": "Berhe",
"Email": "helen@email.com",
"ID_Number": "ET005",
"PhoneNumbers": ["0966778899"]
},
{
"GuestID": 6,
"FirstName": "Yonas",
"LastName": "Alemu",
"Email": "yonas@email.com",
"ID_Number": "ET006",
"PhoneNumbers": ["0977889900"]
},
{
"GuestID": 7,
"FirstName": "Tigist",
"LastName": "Assefa",
"Email": "tigist@email.com",
"ID_Number": "ET007",
"PhoneNumbers": ["0912121212"]
},
{
"GuestID": 8,
"FirstName": "Elias",
"LastName": "Girma",
"Email": "elias@email.com",
"ID_Number": "ET008",
"PhoneNumbers": ["0923232323"]
},
{
"GuestID": 9,
"FirstName": "Rahel",
"LastName": "Solomon",

"Email": "rahel@email.com",
"ID_Number": "ET009",
"PhoneNumbers": ["0934343434"]
},
{
"GuestID": 10,
"FirstName": "Kidus",
"LastName": "Yilma",
"Email": "kidus@email.com",
"ID_Number": "ET010",
"PhoneNumbers": ["0945454545"]
}
]

2. Queries

```
db.guests.insertOne({
 "GuestID": 11,
 "FirstName": "Hermon",
 "LastName": "Daniel",
 "Email": "hermon@email.com",
 "ID_Number": "ET011",
 "PhoneNumbers": ["0988776655"],
 "Status": "VIP"
});
db.guests.find({"FirstName": "Abebe"});
db.guests.updateOne({"GuestID":10}, {$set: {"Email": "kidus_new@email.com"}});
db.guests.deleteOne({"GuestID": 11});
```

3. Results Screenshots

```
_id: ObjectId('6a04a5cae3c2360faa46969f')
GuestID : 1
FirstName : "Abebe"
LastName : "Bikila"
Email : "abebe@email.com"
ID_Number : "ET001"
PhoneNumbers : Array (2)
LoyaltyStatus : "Gold"
```

```
_id: ObjectId('6a04a5cae3c2360faa4696a0')
GuestID : 2
FirstName : "Sara"
LastName : "Mulugeta"
Email : "sara@email.com"
ID_Number : "ET002"
PhoneNumbers : Array (1)
LoyaltyStatus : "Silver"
```

```
_id: ObjectId('6a04a5cae3c2360faa4696a1')
GuestID : 3
FirstName : "Dawit"
LastName : "Isaak"
Email : "dawit@email.com"
ID_Number : "ET003"
PhoneNumbers : Array (1)
```

```
db.guests.insertOne({
  "GuestID": 11,
  "FirstName": "Hermon",
  "LastName": "Daniel",
  "Email": "hermon@email.com",
  "ID_Number": "ET011",
  "PhoneNumbers": ["0988776655"],
  "Status": "VIP"
});
{
  acknowledged: true,
  insertedId: ObjectId('6a04a814bc0273d32b378bda')
}
db.guests.find({"FirstName": "Abebe"});
{
  _id: ObjectId('6a04a5cae3c2360faa46969f'),
  GuestID: 1,
  FirstName: 'Abebe',
  LastName: 'Bikila',
  Email: 'abebe@email.com',
  ID_Number: 'ET001',
  PhoneNumbers: [
    '0911223344',
    '0922112233'
  ],
  LoyaltyStatus: 'Gold'
}
```

```
db.guests.updateOne({"GuestID":10}, {$set: {"Email": "kidus_new@email.com"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
db.guests.deleteOne({"GuestID": 11});
{
  acknowledged: true,
  deletedCount: 1
}
```